SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1410 **COURSE NAME: Compiler Design for Higher Level Processing**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS:4 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I S.MAHALAKSHMI(192524372)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.MAHALAKSHMI

Course Faculty

Annexure A

DESIGN TASK

Q4. Parsing Expression Grammars (PEG) Compared with LL and LR Parsing

A Parsing Expression Grammar, introduced by Bryan Ford in 2004, looks like a CFG but has a different meaning. A PEG rule is a recipe for **recognising** input, not a rule for **generating** strings. Its operators are sequence, **ordered choice** e1 / e2, greedy repetition (*, +, ?), and syntactic predicates (&e "must follow", !e "must not follow") which test the input without consuming it.

1. How PEG resolves ambiguity differently from CFG-based parsers

- **CFG:** a grammar may allow several parse trees for the same string (ambiguity). E → E + E | E * E | id is an example. LL and LR generators detect this as conflicts, and the designer must remove the ambiguity by rewriting the grammar or by adding precedence declarations (as in Q2).
- **PEG:** the alternatives of / are tried strictly left to right, and the first one that succeeds is committed to. A PEG is therefore deterministic by construction, and **ambiguity cannot occur**: every input has at most one parse. The ambiguity is not removed, it is hidden by priority.
- **Consequence:** the order of alternatives is part of the language definition. For example S ← ("a" / "ab") "c" rejects abc: the first alternative "a" succeeds, the choice is committed, and then "c" fails against b. The equivalent CFG S → (a | ab) c accepts abc. PEG never goes back into an alternative that has already succeeded.
- **Benefit:** the common "dangling else" and keyword-versus-identifier problems are solved simply by ordering (put the longer or more specific alternative first). **Risk:** a wrongly ordered rule silently makes some alternative unreachable and no tool reports a conflict.

2. Comparison with LL and LR parsing

Criterion	LL(1) / LL(k)	LR (SLR, LALR, CLR)	PEG / packrat
Parsing direction	Top-down, leftmost derivation	Bottom-up, rightmost derivation in reverse	Top-down, recursive descent with ordered choice
Grammar class	Strict subset of deterministic CFLs. No left recursion; needs left factoring.	All deterministic CFLs (CLR/LR(1)); SLR and LALR cover somewhat fewer grammars. Handles left recursion naturally.	Not a subset of CFG: unlimited lookahead via predicates, can express some non-context-free languages (e.g. $a^n b^n c^n$). Whether every CFL has a PEG is not settled. No direct left recursion.
Ambiguity	Conflicts reported at table build time	Shift/reduce and reduce/reduce conflicts reported	None possible; resolved silently by choice order
Implementation effort	Easy by hand (one function per non-terminal) or by table	Hard by hand; practically needs a generator (yacc/bison) and table construction	Easiest: grammar maps almost directly to code; no separate lexer needed (scannerless)
Time	Linear	Linear	Linear with memoization; exponential in the worst case without it

Criterion	LL(1) / LL(k)	LR (SLR, LALR, CLR)	PEG / packrat
Memory	Stack depth only	Stack plus tables (LALR tables are compact, CLR tables are large)	Memo table proportional to input length times number of rules (can be large)
Backtracking	None (k-token lookahead)	None	Yes: local backtracking on failed alternatives, made cheap by memoization
Error messages	Good: failure point is known	Good, and supports error-recovery productions	Weak: only "failed to match anywhere" unless extra work is done
Typical tools	ANTLR (LL(*)), hand-written parsers	yacc, bison, CUP	LPeg, Peggy (PEG.js), pest, pegen

Relation to SLR, LALR and CLR. All three are LR methods that differ only in how lookaheads are attached to LR(0)/LR(1) items. SLR uses FOLLOW sets (smallest tables, most conflicts). CLR (canonical LR(1)) carries a precise lookahead in each item (most powerful, very large tables). LALR merges CLR states that have the same core (table size of SLR, power close to CLR, and the method used by yacc and bison). PEG sidesteps this whole family of table-conflict analysis by replacing it with committed ordered choice.

3. Real-world adoption: why Python moved to a PEG parser

Python 3.9 (2020) switched to a PEG-based parser generated by **pegen** (PEP 617, by Guido van Rossum, Pablo Galindo and Lysandros Nikolaou), and the old parser was removed in Python 3.10. The reasons given were:

- **LL(1) was too restrictive.** The old pgen parser was LL(1), so the grammar had to be bent to fit one token of lookahead. Constructs that needed more context were handled by special-case code after parsing (for example checking that an assignment target is valid), which made the language definition harder to read and extend.
- **Direct AST construction.** The old pipeline built a concrete syntax tree and then converted it to an AST in a separate hand-written pass. With PEG, each alternative carries an action that builds the AST node directly, removing the intermediate step.
- **Easy to add new syntax.** Features such as parenthesised context managers in with statements and the match statement (with its soft keywords) are much simpler to express with unlimited lookahead and ordered choice than under LL(1).
- **Acceptable performance.** pegen memoizes selected rules. PEP 617 reported speed and memory within about 10% of the old parser, so the cost of packrat-style parsing was small in practice.
- **Left recursion is supported by pegen** (using the seed-growing technique of Warth et al.), so the natural left-recursive rules for binary operators can be written directly. Python also uses a second pass with special invalid_ rules to produce better syntax-error messages.

Other PEG users include Lua (LPeg), Rust (pest) and JavaScript (Peggy/PEG.js), mostly for DSLs, configuration formats and markup. By contrast, major production compilers still use hand-written recursive descent (GCC, Clang, javac) or LALR generators (PHP and Ruby have used Bison).

4. Limitations of PEG

- **Left recursion.** A rule such as E ← E "+" T / T recurses forever in a plain PEG parser. Workarounds are to rewrite it as a loop E ← T ("+" T)* (which loses the natural left-associative tree shape unless the actions rebuild it) or to use a left-recursion-aware algorithm (as pegen does), which adds complexity.

- **Ordered choice hides mistakes.** Because the first match wins, a prefix alternative placed before a longer one makes the longer one unreachable ("a" / "ab"). There is no conflict report and no ambiguity checker, so errors appear as wrong behaviour on particular inputs.
- **No backtracking into a succeeded choice, and greedy repetition.** a* a can never match, because a* consumes all the a's and never gives one back. Grammars cannot always be written as a naive transliteration of a CFG.
- **Poor error reporting.** A failed alternative leaves no information about what was expected; reporting the furthest failure position and expected set requires extra machinery.
- **Memory and predictability.** Packrat memoization trades space for time; for large inputs the memo table can be much larger than an LR stack. Without memoization, pathological grammars can take exponential time.
- **Weak formal tooling.** There is no standard way to check that a grammar defines the language the author intended, and grammars are harder to compose than CFGs, because the meaning of a rule depends on its alternatives' order.

5. Conclusion: is PEG suitable for all programming languages?

No. PEG is a strong choice for specific cases, not a universal replacement.

- **Good fit:** languages whose grammar evolves often or needs unbounded lookahead (such as Python), DSLs and data formats, and scannerless parsing where lexical and syntactic rules are mixed. Here the simple implementation, the lack of a separate lexer, and direct AST actions outweigh the costs.
- **Poor fit:** languages that need a formal guarantee that the grammar is unambiguous (the LR conflict report provides one and PEG does not), very large inputs or tight memory budgets, and tools that need strong error recovery (IDEs, compilers that must report many errors per run). LALR/LR generators, or hand-written recursive descent with custom error handling, remain better in those settings, which is why mainstream C, C++ and Java compilers have not moved to PEG.

Justification: PEG trades the analysis power of CFG parsing (ambiguity detection and conflict reports) for simplicity and expressive lookahead. That trade pays off when the grammar is designed and maintained by people who can test it thoroughly, and does not pay off when correctness must be proved from the grammar alone. Python's adoption shows it works well at scale for one language; it does not show it is right for all.

Q5. Advanced LL(1) Parser Evaluation and Design

Given grammar

> S → S + T | T
>
> T → T * F | F
>
> F → (S) | id

1. Why the given grammar is not LL(1)

- **Left recursion:** S → S + T and T → T * F are directly left-recursive. A top-down parser expanding S would call S forever without reading any input.
- **FIRST/FIRST conflict:** for S, both alternatives S + T and T have FIRST = { (, id }, so one lookahead token cannot choose between them. The same is true for T.

2. Eliminating left recursion

Using $A \to A\alpha \mid \beta \implies A \to \beta\ A',\ A' \to \alpha\ A' \mid \varepsilon$:

Non-terminal	α	β	Result
S	+ T	T	S → T S' and S' → + T S' | ε
T	* F	F	T → F T' and T' → * F T' | ε

Left factoring. Left factoring is needed when two alternatives of one non-terminal begin with the same symbols. Checking every non-terminal of the new grammar, no two alternatives share a prefix, so the step changes nothing here:

- S': + T S' and ε. No common prefix.
- T': * F T' and ε. No common prefix.
- F: (S) begins with (, id begins with id. No common prefix.

Redesigned LL(1) grammar:

> (1) S → T S'

(2) S' → + T S'

(3) S' → ε

(4) T → F T'

(5) T' → * F T'

(6) T' → ε

(7) F → (S)

(8) F → id

3. FIRST and FOLLOW sets

Non-terminal	FIRST	FOLLOW
S	{ (, id }	{ $,) }
S'	{ +, ε }	{ $,) }
T	{ (, id }	{ +, $,) }
T'	{ *, ε }	{ +, $,) }
F	{ (, id }	{ *, +, $,) }

Working for FOLLOW:

- FOLLOW(S) = { $ } ∪ {) } because S is the start symbol and F → (S) places) after S.
- FOLLOW(S') = FOLLOW(S) = { $,) } from S → T S' and S' → + T S' (S' is last).
- FOLLOW(T) = (FIRST(S') − {ε}) ∪ FOLLOW(S) ∪ FOLLOW(S') = { +, $,) }, since S' is nullable.
- FOLLOW(T') = FOLLOW(T) = { +, $,) } (T' is last in T → F T' and T' → * F T').
- FOLLOW(F) = (FIRST(T') − {ε}) ∪ FOLLOW(T) ∪ FOLLOW(T') = { *, +, $,) }.

4. LL(1) predictive parsing table

	id	**+**	*****	**(**	**)**	**$**
S	S → T S'			S → T S'		
S'		S' → + T S'			S' → ε	S' → ε
T	T → F T'			T → F T'		
T'		T' → ε	T' → * F T'		T' → ε	T' → ε
F	F → id			F → (S)		

5. Does the grammar satisfy the LL(1) conditions?

A grammar is LL(1) if, for every non-terminal A with alternatives A → α | β, (i) FIRST(α) ∩ FIRST(β) = ∅, and (ii) if ε ∈ FIRST(β) then FIRST(α) ∩ FOLLOW(A) = ∅ (and neither alternative has ε in its FIRST set when both are nullable).

Non-terminal	**Alternatives**	**Check (i)**	**Check (ii)**	**Verdict**
S'	+ T S' | ε	{ + } ∩ { ε } = ∅	{ + } ∩ FOLLOW(S') = { + } ∩ { $,) } = ∅	OK
T'	* F T' | ε	{ * } ∩ { ε } = ∅	{ * } ∩ { +, $,) } = ∅	OK
F	(S) | id	{ (} ∩ { id } = ∅	no ε alternative	OK
S, T	single alternative	not applicable	not applicable	OK

Justification: every pair of alternatives has disjoint FIRST sets, and the only nullable alternatives (S' → ε and T' → ε) do not conflict with FOLLOW. The grammar is not left-recursive and no table cell has two entries. **Therefore the redesigned grammar is LL(1).**

6. Parsing id + id * id

Step	Stack (top at left)	Remaining input	Parsing action
1	S $	id + id * id $	S → T S'
2	T S' $	id + id * id $	T → F T'
3	F T' S' $	id + id * id $	F → id
4	id T' S' $	id + id * id $	Match id
5	T' S' $	+ id * id $	T' → ε
6	S' $	+ id * id $	S' → + T S'
7	+ T S' $	+ id * id $	Match +
8	T S' $	id * id $	T → F T'
9	F T' S' $	id * id $	F → id
10	id T' S' $	id * id $	Match id
11	T' S' $	* id $	T' → * F T'
12	* F T' S' $	* id $	Match *
13	F T' S' $	id $	F → id
14	id T' S' $	id $	Match id
15	T' S' $	$	T' → ε
16	S' $	$	S' → ε
17	$	$	Accept

Leftmost derivation:

> S ⇒ T S' ⇒ F T' S' ⇒ id T' S' ⇒ id S' ⇒ id + T S' ⇒ id + F T' S'
> ⇒ id + id T' S' ⇒ id + id * F T' S' ⇒ id + id * id T' S'
> ⇒ id + id * id S' ⇒ id + id * id

Result: both the stack and the input reach $, so the string id + id * id is ACCEPTED. The derivation also shows that * is expanded inside T, below +, so multiplication takes precedence over addition.

Error check (for completeness): for the invalid input id + * id, after matching id and + the stack top is T and the lookahead is *. M[T, *] is blank, so the parser reports a syntax error at *.

Q13. Compiler Front-End for a Simple Expression Language

Goal. Design the front-end of a compiler for assignment statements over arithmetic expressions: a lexical analyser, an LL(1) parser, a syntax-tree builder and a three-address-code generator, with syntax-error handling. Test input: a = b + c * d.

Phase	Input	Output	Component
Lexical analysis	Character stream	Token stream	Scanner (finite automaton)
Syntax analysis	Token stream	Parse (derivation) and syntax tree	LL(1) predictive parser
Semantic analysis	Syntax tree	Checked tree, symbol table	Symbol table
Intermediate code generation	Syntax tree	Three-address code	Tree walker

1. Lexical analyser: token specification

Token	Pattern (regular expression)	Examples
ID	[A-Za-z_][A-Za-z0-9_]*	a, b, count, x1
NUM	[0-9]+(\.[0-9]+)?	10, 3.14
ASSIGN	=	=
PLUS / MINUS	+ / -	+ -
MUL / DIV	* / /	* /
LPAREN / RPAREN	(/)	( )
WHITESPACE	[\t\n]+	skipped, no token
EOF	end of input	$

The scanner is a deterministic finite automaton that applies the **longest-match rule**: it starts in an initial state; a letter or underscore moves to the identifier state, which loops on letters/digits/underscore; a digit moves to the number state (loops on digits, optionally one . followed by digits); a single operator character is accepted immediately. When no transition is possible the longest accepted lexeme is returned as a token. A character that starts no token is a **lexical error**. Identifier lexemes are entered in the symbol table.

Token stream for a = b + c * d:

#	Lexeme	Token	Attribute
1	a	ID	pointer to symbol-table entry for a
2	=	ASSIGN	-
3	b	ID	entry for b
4	+	PLUS	-
5	c	ID	entry for c
6	*	MUL	-
7	d	ID	entry for d
8	(end)	EOF	$

2. Grammar design and parser choice

An LL(1) predictive parser is chosen because the language is small, the grammar can be made LL(1) easily, it is simple to implement, and it gives precise error positions. Precedence and associativity are built into the grammar: * and / are handled at a lower level (T) than + and - (E), so they bind tighter; both are left-associative.

Grammar (already free of left recursion and common prefixes):

> (1) Stmt → id = E

(2) E → T E'

(3) E' → + T E' | - T E' | ε

(4) T → F T'

(5) T' → * F T' | / F T' | ε

(6) F → (E) | id | num

Non-terminal	FIRST	FOLLOW
Stmt	{ id }	{ $ }
E	{ (, id, num }	{ $,) }
E'	{ +, -, ε }	{ $,) }
T	{ (, id, num }	{ +, -, $,) }
T'	{ *, /, ε }	{ +, -, $,) }
F	{ (, id, num }	{ *, /, +, -, $,) }

LL(1) parsing table (rows are non-terminals, columns are the lookahead token; blank means error):

	id	**num**	**=**	**+**	**-**	*****	**/**	**(**	**)**	**$**
Stmt	1									
E	2	2						2		
E'				+TE'	-TE'				ε	ε
T	4	4						4		
T'				ε	ε	*FT'	/FT'		ε	ε
F	id	num						(E)		

Entries 1, 2 and 4 stand for the productions Stmt → id = E, E → T E' and T → F T' (the full right sides are listed above). Each cell has at most one entry, so the grammar is LL(1).

3. Predictive parsing of a = b + c * d

Tokens are written by their lexemes in the input column; the stack holds token classes (id).

Step	Stack (top at left)	Remaining input	Parsing action
1	Stmt $	a = b + c * d $	Stmt → id = E
2	id = E $	a = b + c * d $	Match id (a)
3	= E $	= b + c * d $	Match =
4	E $	b + c * d $	E → T E'
5	T E' $	b + c * d $	T → F T'
6	F T' E' $	b + c * d $	F → id
7	id T' E' $	b + c * d $	Match id (b)
8	T' E' $	+ c * d $	T' → ε
9	E' $	+ c * d $	E' → + T E'
10	+ T E' $	+ c * d $	Match +
11	T E' $	c * d $	T → F T'
12	F T' E' $	c * d $	F → id
13	id T' E' $	c * d $	Match id (c)
14	T' E' $	* d $	T' → * F T'
15	* F T' E' $	* d $	Match *
16	F T' E' $	d $	F → id
17	id T' E' $	d $	Match id (d)

Step	Stack (top at left)	Remaining input	Parsing action
18	T' E' $	$	T' → ε
19	E' $	$	E' → ε
20	$	$	Accept

Result: the statement is syntactically correct (ACCEPTED).

4. Syntax tree

The parse tree contains every grammar symbol, so a compact **abstract syntax tree** is built by semantic actions attached to the productions. Because the grammar is right-recursive after left-recursion removal, the tree for E' and T' is built with an inherited attribute that carries the left operand, so left associativity is preserved.

Production	Semantic action (attribute rules)
E → T E'	E'.inh = T.node; E.node = E'.syn
E' → + T E'₁	E'₁.inh = mknode('+', E'.inh, T.node); E'.syn = E'₁.syn
E' → ε	E'.syn = E'.inh
T → F T'	T'.inh = F.node; T.node = T'.syn
T' → * F T'₁	T'₁.inh = mknode('*', T'.inh, F.node); T'.syn = T'₁.syn
T' → ε	T'.syn = T'.inh
F → id	F.node = mkleaf(id, symbol-table entry)
Stmt → id = E	Stmt.node = mknode('=', mkleaf(id), E.node)

Syntax tree for a = b + c * d:

```
      =
     / \
    a   +
```

```
    / \
   b   *
      / \
     c   d
```

The node * is a child of +, so c * d is evaluated first. A pre-order listing is = a + b * c d.

5. Intermediate code: three-address code

A post-order walk of the syntax tree visits the operands before the operator. For each interior node a fresh temporary tN is created and one instruction of the form result = operand1 op operand2 is emitted. The assignment node copies the final temporary into the target variable.

Tree node visited	Value returned	Instruction emitted
leaf c	c	-
leaf d	d	-
node * (c, d)	t1	t1 = c * d
leaf b	b	-
node + (b, t1)	t2	t2 = b + t1
node = (a, t2)	a	a = t2

Three-address code produced:

```
t1 = c * d
t2 = b + t1
a  = t2
```

Quadruple representation:

#	op	arg1	arg2	result
0	*	c	d	t1
1	+	b	t1	t2
2	=	t2	-	a

Triple representation (results referred to by instruction number):

#	op	arg1	arg2
(0)	*	c	d
(1)	+	b	(0)
(2)	=	a	(1)

6. Symbol table and semantic checks

Name	Kind	Type (assumed)	Role in this statement
a	variable	int	assigned (left-hand side)
b	variable	int	operand (right-hand side)
c	variable	int	operand (right-hand side)
d	variable	int	operand (right-hand side)
t1, t2	temporary	int	compiler-generated

This small language has a single numeric type, so the semantic checks are: each identifier on the right-hand side must be known to the symbol table (otherwise report "undeclared variable"), and the left-hand side must be a variable.

7. Syntax-error handling

Lexical errors: an illegal character is reported with its position and skipped, and scanning continues.

Syntax errors (panic-mode recovery in the table-driven parser): when the stack top is a non-terminal A and M[A, a] is blank, report the error with the line, the offending token and the expected tokens. Then: (a) if a ∈ FOLLOW(A), pop A (treat it as if it derived ε) so that parsing can resynchronise; (b) otherwise skip the input token a and retry. When the stack top is a terminal that does not match the input, report it and pop the terminal (as if it had been inserted). Parsing then continues, so that several errors can be reported in one run.

Input	Where detected	Diagnostic	Recovery
a = b + * c	T on stack, lookahead *; M[T, *] blank	Syntax error: unexpected '*', expected identifier, number or '('	* ∉ FOLLOW(T): skip *, then continue with c
a = (b + c * d	Terminal) on stack, lookahead $	Syntax error: missing ')' before end of input	Pop) (assume inserted)
a = b + c d	T' on stack, lookahead d; M[T', id] blank	Syntax error: unexpected 'd', expected an operator or end of statement	Skip d
a = b + c *	F on stack, lookahead $	Syntax error: unexpected end of input, expected an operand	$ ∈ FOLLOW(F): pop F and stop
a b + c	Terminal = on stack, lookahead b	Syntax error: expected '=' after 'a'	Pop =, continue with E
a = b @ c	Scanner	Lexical error: illegal character '@' at position 6	Discard @, continue

8. Complete compilation of a = b + c * d

Phase	Result
Source program	a = b + c * d
1. Lexical analysis	<id,a> <=> <id,b> <+> <id,c> <*> <id,d> <$>
2. Syntax analysis	LL(1) parse succeeds in 20 steps (table in section 3). No syntax error.
3. Syntax tree	=(a, +(b, *(c, d))): multiplication is nested under addition
4. Semantic analysis	Symbol table holds a, b, c, d; all uses valid
5. Intermediate code	t1 = c * d, t2 = b + t1, a = t2
Output	Three-address code handed to the optimiser and code generator (the back-end)

Result: the front-end accepts the statement and produces the three-address code above. The program in the Appendix implements this same pipeline and produced exactly these outputs when run.

Appendix. Prototype Front-End (Python) and Its Output

The program below is a recursive-descent form of the LL(1) parser (the loops in expr and term correspond to E' and T'). It scans, parses, builds the syntax tree and emits three-address code.

```
import re

# ---------- Q13: lexer, LL(1) parser, AST, TAC ----------
TOKEN_RE = re.compile(r'\s*(?:(?P<num>\d+(?:\.\d+)?)|(?P<id>[A-Za-z_]\w*)|(?P<op>[=+\-*/()]))')
def lex(src):
    pos = 0; out = []
    while pos < len(src):
        if src[pos].isspace(): pos += 1; continue
        m = TOKEN_RE.match(src, pos)
        if not m: raise SyntaxError(f"Lexical error: illegal character {src[pos]!r} at position {pos}")
        pos = m.end()
        if m.group('num'): out.append(('num', m.group('num')))
        elif m.group('id'): out.append(('id', m.group('id')))
        else: out.append((m.group('op'), m.group('op')))
    out.append(('$', '$'))
    return out

class Parser:
    def __init__(self, toks):
        self.t = toks; self.i = 0; self.n = 0; self.code = []; self.errors = []
    def peek(self): return self.t[self.i][0]
    def eat(self, k):
        if self.peek() == k: self.i += 1; return self.t[self.i-1]
```

```
        raise SyntaxError(f"expected '{k}' but found '{self.t[self.i][1]}'")
    def newt(self): self.n += 1; return f't{self.n}'
    def stmt(self):
        a = self.eat('id'); self.eat('=')
        rhs = self.expr()
        self.eat('$')
        return ('=', ('id', a[1]), rhs)
    def expr(self):
        l = self.term()
        while self.peek() in ('+', '-'):
            op = self.eat(self.peek())[0]; r = self.term(); l = (op, l, r)
        return l
    def term(self):
        l = self.factor()
        while self.peek() in ('*', '/'):
            op = self.eat(self.peek())[0]; r = self.factor(); l = (op, l, r)
        return l
    def factor(self):
        k = self.peek()
        if k == 'id': return ('id', self.eat('id')[1])
        if k == 'num': return ('num', self.eat('num')[1])
        if k == '(':
            self.eat('('); e = self.expr(); self.eat(')'); return e
        raise SyntaxError(f"unexpected '{self.t[self.i][1]}', expected id, number or '('")

def gen(node, code, cnt):
    if node[0] in ('id', 'num'): return node[1]
    if node[0] == '=':
        r = gen(node[2], code, cnt); code.append((node[1][1], r, None, None)); return
node[1][1]
```

```
    l = gen(node[1], code, cnt); r = gen(node[2], code, cnt)
    cnt[0] += 1; t = f't{cnt[0]}'
    code.append((t, l, node[0], r)); return t

def show(n, d=0):
    if n[0] in ('id', 'num'): return '  '*d + n[1]
    return '  '*d + n[0] + '\n' + show(n[1], d+1) + '\n' + show(n[2], d+1)
```

Output when run on a = b + c * d:

```
Source   : a = b + c * d
Tokens   : <id,a> <=> <id,b> <+> <id,c> <*> <id,d> <$>
Syntax tree (pre-order, indented):
=
  a
  +
    b
    *
      c
      d
Three-address code:
  t1 = c * d
  t2 = b + t1
  a = t2
```

CO2 Assessment Tool-1

Name:- S.Mahalakshmi

Regno:- 192524372

Course:- Compiler Design

Course code:- CSA1410

Slot:- D

Design Task

1) Predictive parser Design:

Task:- Construct an LL(1) predictive parsing table for the given grammar and demonstrate its application by Parsing the specified input string step-by-step, clearly showing stack operations, input symbols, and parsing actions:

Solutions:-

Grammar:

$E \rightarrow TE'$

$E' \rightarrow +TE' | \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' | \varepsilon$

$F \rightarrow (E) | id$

The grammar has no left recursion, so nothing needs removing.

FIRST:-

- FIRST(E) = { (, id }
- First[E') = { +, ε }
- FIRST(T') = { *, ε }

FOLLOW:-

- FOLLOW(E) = { $,) }
- FOLLOW(T) = { +, $,) }

·) FOLLOW(F) = { *, +, $,) }

Parsing table :-

	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \varepsilon$	$E' \to \varepsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \varepsilon$	$T' \to *FT'$		$T' \to \varepsilon$	$T' \to \varepsilon$
F	$F \to id$			$F \to (E)$		

Parsing : id + id * id

Stack	Input	Action
$E\$$	id + id * id $	$E \to TE'$
$TE'\$$	id + id * id $	$T \to FT'$
$FT'E'\$$	id + id * id $	$F \to id$
$idT'E'\$$	id + id * id $	match id
$T'E'\$$	+ id * id $	$T' \to \varepsilon$
$E'\$$	+ id * id $	$E' \to +TE'$
$+TE'\$$	+ id * id $	match +
$TE'\$$	id * id $	$T \to FT'$
$FT'E'\$$	id * id $	$F \to id$
$idT'E'\$$	id * id $	match id
$T'E'\$$	* id $	$T' \to *FT'$
$*FT'E'\$$	* id $	match *
$FT'E'\$$	id $	$F \to id$
$idT'E'\$$	id $	match id
$T'E'\$$	$	$T' \to \varepsilon$
$E'\$$	$	$E' \to \varepsilon$
$\$$	$	Accept

2) Shift - Reduce Parsing:-

Task:-

Apply shift reduce parsing to the given grammar and input string and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions, clearly indicating all reductions and decisions made during parsing.

Solutions:-

Grammar:-

$E \rightarrow E+E^*$

$E \rightarrow E*E$

$E \rightarrow id$

Input String:-

id + id * id

Parsing Table:-

Steps	Stack	Input	Action
1)	$	id + id * id $	Shift
2)	$id	+ id * id $	Reduce E→id
3)	$E	+ id * id $	Shift
4)	$E +	id * id $	Shift
5)	$E + id	* id $	Reduce E→id
6)	$E + E	* id $	Shift (* has higher precedence than +)
7)	$E + E *	id $	~~Reduce E→id~~ Shift

Steps	Stack	Input	Action
8	$E+E*id	$	Reduce E→id
9	$E+E*E	$	Reduce E→E*E
10	$E+E	$	Reduce E→E+E
11	$E	$	Accept